

ASSIGNMENT 5: GRAPH ALGORITHMS

Instructor: Orhan Özgüner

Due: December 5 before 11:59 PM

Problem 1

There are n basketball teams in the world. The ranking of these teams from the previous year is available. This year, some of these n teams played against each other and the winner of each game was determined. There were m games in total. The International Basketball Association wants to introduce a new performance criterion, called “domination factor”, defined as follows: Team i is said to “dominate” team j if we can find a chain of games such that j was beaten by a team that was beaten by a team that was beaten by a team ... that was beaten by i (observe that, according to this definition, domination can be bi-directional, i.e., i and j can dominate each other). Then, for each team i , the domination factor z_i is defined as the rank of the best team (that is, the highest ranked team according to last year’s rankings) that is dominated by team i . Describe an efficient algorithm to compute the domination factor for all the n teams.

Pseudocode

Algorithm 1 Compute Domination Factor with Revised Graph Structure

```

1: procedure COMPUTEDOMINATIONFACTOR( $V, E$ )
2:    $d\_factor \leftarrow$  array same size of  $V$ , containing FALSE
3:    $visited \leftarrow$  array same size as  $V$ , containing  $\infty$ 
4:   for  $vert \in V$  do                                      $\triangleright$  Iterate in ascending order of rank
5:     if  $\neg visited[vert]$  then
6:       DFS( $vert, vert, E, d\_factor, visited$ )
7:     end if
8:   end for
9:   return  $d\_factor$ 
10: end procedure
11: procedure DFS( $current, start, E, dominance, visited$ )
12:   for ( $current, next$ )  $\in E[current]$  do                  $\triangleright$  For each edge coming from the current node...
13:     if  $\neg visited[next]$  then
14:        $visited[next] \leftarrow \mathbf{TRUE}$ 
15:        $dominance[next] \leftarrow start$ 
16:       DFS( $next, start, E, dominance, visited$ )
17:     end if
18:   end for
19: end procedure
  
```

Runtime Analysis

Although the DFS function is recursive and called repeatedly from inside a for-loop, it only visits each vertex and edge *once*. Therefore, the algorithm runs in $\Theta(V + E)$ time and $\Theta(V)$ space.

Proof of Correctness

Proof. Let $G = (V, E)$ represent a directed graph where V is the set of teams and $E \subseteq V \times V$ represents the domination relations. The proof is structured in two parts: showing that the algorithm visits all teams (vertices), and establishing the correctness of the domination factor at each vertex.

Part 1: Every Team is Visited and Forms a Valid DFS Subtree.

We perform mathematical induction on the number of vertices in G .

Base Case: For $|V| = 1$, there exists a single vertex v . Trivially, v is visited, and the DFS subtree T rooted at v contains exactly one vertex, v , therefore it is valid.

Inductive Step: Assume the proposition holds for all graphs of size k , and consider a graph $G' = (V', E')$ with $|V'| = k + 1$. Remove a vertex $t \in V'$ to produce a graph G that is of size k . Our induction assumes that a DFS on G forms valid DFS subtrees for each vertex and their respective neighbors. Vertex t either initiates a new DFS call or is reached from an existing subtree in G . In both cases, the DFS process ensures that t and all its neighbors in G' are visited, maintaining the validity of the DFS subtree structure for G' .

Part 2: Correctness of Domination Factor.

Define $D : V \leftarrow N \cup \{\infty\}$ such that $D(i)$ is the domination factor of team i , initialized to ∞ .

Correct Assignment: If a path $P : j \leftarrow i$ in G , where $P \subseteq E$, exists, then j dominates i . The assignment $D(i) = j$ during DFS reflects the existence of P , establishing a correct domination relation.

Optimality of Assignment: Assume $\exists k \in V$, such that k has a higher rank than $D(i)$ and dominates i , i.e., $\exists P : k \leftarrow i$. In DFS, vertices are visited in ascending order of rank, so k will precede i in the traversal. The existence of P would imply $D(i)$ would be updated to k , contradicting the assumption that $D(i)$ is not the highest possible rank of a team dominating i .

So, the algorithm correctly assigns to each team the highest rank of a team it can dominate. $\square$

Problem 2

Prove or disprove the following statements:

- (a) Let $G = (V, E)$ be a directed graph. For any $uv \in E$, if some run of Depth-First-Search (DFS) on G results in $v.f > u.f$, then uv must be on a cycle.

Solution

This statement is **true**.

Proof. Suppose that $uv \in E$ has $v.f > u.f$ after a DFS run on G . The parenthesis theorem allows two potential relationships between the intervals $[u.d, u.f]$ and $[v.d, v.f]$: either they are entirely disjoint, or one is entirely contained within the other.

Case 1: Disjoint Intervals (Impossible). If $[u.d, u.f]$ is disjoint from $[v.d, v.f]$, then either $u.f < v.d$ or $v.f < u.d$. However, since $v.f > u.f$, the former is not possible. For the latter, if $v.f < u.d$, then u was not discovered when v was finalized, which could not occur—we know there exists an edge uv , so the DFS would have explored that edge before finalizing v . Therefore, this case is impossible.

Case 2: $[u.d, u.f]$ contained within $[v.d, v.f]$. In this scenario, u is a descendant of v in the DFS tree. “[$u.d, u.f$] contained within $[v.d, v.f]$ ” implies that $v.d < u.d < u.f < v.f$. Therefore, there exists a path from v to u in the DFS tree. Adding the edge uv , creates a cycle containing uv .

Case 3: $[v.d, v.f]$ contained within $[u.d, u.f]$ (Impossible). If $[v.d, v.f]$ is entirely contained within $[u.d, u.f]$, this would mean that $u.d < v.d < v.f < u.f$. However, this contradicts our initial assumption that $v.f > u.f$. So, this case is also impossible.

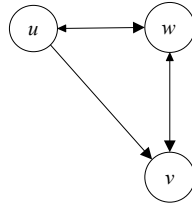
Since Cases 1 and 3 are impossible under the assumptions made, the only viable scenario is Case 2, where the existence of uv with $v.f > u.f$ implies the existence of a cycle in G that includes uv . $\square$

- (b) Consider any run of DFS on a directed graph $G = (V, E)$. For any edge $uv \in E$, if there is a path from v to u in G , then uv cannot be a cross edge.

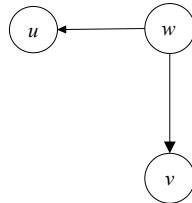
Solution

This statement is **false**.

Counterexample. Suppose we have a graph $G = (V, E)$ where $V = \{u, v, w\}$ and $E = \{uv, vw, uv, wu, uw\}$. The graph looks like this:



There is an edge uv , and a path from v to u that passes through w . Suppose we run a DFS starting at vertex w . If the traversal order is $w \rightarrow v \rightarrow u$, we will form the following DFS tree:



Therefore, even though there is a path from v to u in the graph G , the edge uv is still a cross edge.

Problem 3

There are two types of professional wrestlers: “babyfaces” (“good guys”) and “heels” (“bad guys”). Between any pair of professional wrestlers, there may or may not be a rivalry. Suppose we have n professional wrestlers and we have a list of r pairs of wrestlers for which there are rivalries. Give an efficient algorithm that determines whether it is possible to designate some of the wrestlers as babyfaces and the remainder as heels such that each rivalry is between a babyface and a heel. If it is possible to perform such a designation, your algorithm should produce it.

Pseudocode

Runtime Analysis

This algorithm visits each rival edge for each node, once. So, the algorithm runs in $\Theta(V + E)$ time and requires $\Theta(V)$ space.

Proof of Correctness

Correctness:

Proof. Consider the graph $G = (V, E)$ where V represents wrestlers and E represents rivalries. We define two disjoint subsets of V , denoted V_b for babyfaces and V_h for heels, such that $V_b \cup V_h = V$ and $V_b \cap V_h = \emptyset$.

Algorithm 2 Determine Wrestler Designations

```
1: procedure DESIGNATEWRESTLERS( $n, R$ )
2:    $group \leftarrow$  array of size  $n$  containing NULL
3:    $visited \leftarrow$  array of size  $n$  containing FALSE
4:    $q \leftarrow$  empty queue
5:   for  $i \leftarrow 1$  to  $n$  do
6:     if  $group[i] = \mathbf{NULL}$  then
7:        $group[i] \leftarrow \mathbf{BABYFACE}$ 
8:        $visited[i] \leftarrow \mathbf{TRUE}$ 
9:       Enqueue  $i$  in  $q$ 
10:    while  $q$  is not empty do
11:       $current \leftarrow$  last element in  $q$ 
12:      for  $rival$  in  $R[current]$  do
13:        if  $group[rival] = \mathbf{NULL}$  then
14:          if  $group[current] = \mathbf{HEEL}$  then
15:             $group[rival] \leftarrow \mathbf{BABYFACE}$ 
16:          else
17:             $group[rival] \leftarrow \mathbf{HEEL}$ 
18:          end if
19:           $visited[rival] = \mathbf{TRUE}$ 
20:          Enqueue  $rival$  in  $q$ .
21:        end if
22:        else if ( $group[rival] = group[current]$ ) then
23:          return NO SOLUTION
24:        end for
25:      end while
26:    end if
27:  end for
28:  return  $group$ 
29: end procedure
```

The algorithm employs BFS to assign each vertex (wrestler) in V to either V_b or V_h . For a vertex $v \in V$, all adjacent vertices (rivals) are assigned to the opposite subset. This process is executed iteratively for each unvisited vertex.

If a vertex v is assigned to V_b , then all adjacent vertices u where $(v, u) \in E$ are assigned to V_h , and vice versa. If an adjacent vertex is already assigned to the same subset, the algorithm declares the designation impossible, indicating a contradiction in bipartiteness. So, if the algorithm completes without contradiction, it guarantees that for every $(u, v) \in E$, either $u \in V_b$ and $v \in V_h$, or $u \in V_h$ and $v \in V_b$. This condition aligns with the definition of a bipartite graph, validating the correctness of the assignment. $\square$

Completeness:

Proof. The algorithm iteratively applies a BFS-like strategy starting from each unvisited vertex in V , exploring all the connected components of G . In each connected component, the algorithm assigns vertices to either V_b or V_h , ensuring that each component is bipartite. A graph G is bipartite if and only if it is possible to divide V into two disjoint subsets such that every edge connects a vertex in one subset to a vertex in the other. Therefore, if a valid babyface-heel designation exists in G , then G must be bipartite. Since the algorithm explores all connected components and assigns vertices to create bipartite subgraphs, if a valid designation exists, the algorithm will correctly identify and assign it. Thus, the algorithm is complete in its ability to find a valid designation if one exists. $\square$